

LAB3 실험 전 보고서

주변장치 제어 7종 — PWM · 피에조 · 스텝 모터 · 시계 · 문자 LCD · UART

이름

형종희

학번

2023440145

조

C조

작성일

2026-09-27

대상 보드

Combo II-DLD S75 ·
xc7s75fgga484-1

도구

Vivado 2026.1 · Icarus
Verilog 12.0

GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
구조	LAB1/ · LAB2/ · LAB3/ 가 형제 폴더
LAB3 설계 소스 커밋	d1479db — 7개 회로의 RTL · 테스트벤치 · XDC (30개 파일)
LAB3 경로	LAB3/circuits/LAB3-01~07/ , LAB3/evidence/ , LAB3/LAB3_실험전보고서.html · .pdf
실험 전 보고서 태그	lab3-prelab — 이 보고서(HTML · PDF)가 들어간 커밋을 가리킴. 최종 제출 태그 lab3-submit 은 실험 후 보고서 커밋에 따로 건다.
회로 번호	01 LED PWM · 02 RGB PWM · 03 피에조 · 04 스텝 모터 · 05 MM:SS 시계 · 06 Character LCD · 07 UART 에코

0. 7개 회로 사전 결과 요약

7개 회로 모두 RTL 작성 → 두 시뮬레이터(Vivado XSim, Icarus)의 자기검사 통과 → 합성 · 구현 · 비트스트림 생성까지 마쳤다. 남은 일은 보드 실습 전 새 장치 핀 확인과 보드 동작 확인이다.

#	회로	XSim · Icarus	LUT	FF	IOB	WNS (ns)	추가 핀 확인
1	LED PWM	checks=4	92	52	11	8.971	불필요
2	RGB PWM	checks=2	276	144	17	8.629	RGB 12핀
3	피에조	edges=5	21	18	3	16.761	Y21
4	스텝 모터	checks=8	20	25	8	16.252	4핀
5	MM:SS 시계	checks=7	83	58	18	15.118	불필요
6	Character LCD	bytes=40	111	93	13	14.220	11핀
7	UART 에코	checks=3	134	118	12	14.617	2핀

공통 설계 규칙

- **클록**: 메인 클록 B6, 보드 다이얼 **F = 50 MHz**. 7개 XDC 모두 `create_clock -period 20.000`.
- **리셋**: 키패드 1(K4), 누르면 High → 비동기 리셋 `rst_p`.
- **시간 만들기**: 원하는 시간 t 는 클록 수 $n = t \cdot f_{clk}$ 를 세서 만든다. 예) 20 ms = 1,000,000클록, 1 s = 50,000,000클록.
- **시뮬레이션 축소**: 테스트벤치는 CLK_HZ 같은 파라미터를 작게 바꿔 같은 동작을 수천~수만 배 빨리 확인한다. RTL은 바뀌지 않는다.

핀 확인 대기 — 대상: 새 장치(RGB · 피에조 · 스텝 모터 · LCD · UART)의 핀 30개. 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

목차

1. 실험 1 · LED 밝기 조절 (PWM)
2. 실험 2 · RGB LED 색 조합 (3채널 PWM)
3. 실험 3 · 피에조 부저 (주파수 분주로 음 만들기)
4. 실험 4 · 스텝 모터 (2상 여자 순서 만들기)
5. 실험 5 · MM:SS 디지털 시계 (7세그먼트 4자리)
6. 실험 6 · Character LCD (HD44780 호환 16×2)
7. 실험 7 · UART 에코 (9600 bps, 8N1)

1. 실험 1 · LED 밝기 조절 (PWM)

① 목적

버튼을 누를 때마다 LED 8개의 밝기가 0 → 10% → ... → 100% → 0으로 한 단계씩 바뀌게 한다. LED는 켜짐 · 꺼짐 두 상태만 있지만, 1초에 1,000번씩 "켜진 시간의 비율"을 바꾸면 눈에는 밝기가 바뀐 것처럼 보인다. 선풍기 날개가 빨리 돌면 하나의 원판처럼 보이는 것과 같은 원리다.

② 이론 · 설계 계산

한 PWM 주기의 클록 수 N 과, 밝기 단계 L ($0 \leq L \leq 10$)에서 High를 유지하는 클록 수 T_{on} 은 다음과 같다.

$$N = \frac{f_{clk}}{f_{PWM}} = \frac{50\,000\,000}{1\,000} = 50\,000, \quad T_{on} = \frac{N \cdot L}{10} = 5\,000 L$$
$$D = \frac{T_{on}}{N} = \frac{L}{10} \quad (L = 3 \Rightarrow D = 30\%)$$

버튼은 기계 접점이라 누르는 순간 수 ms 동안 0과 1을 오간다(채터링). `button_onepulse` 는 2단 플립플롭으로 비동기 입력을 클록에 맞춘 뒤, 값이 **1 000 000** 클록 = **20 ms** 동안 그대로일 때만 한 클록짜리 펄스를 낸다.

파라미터	값	의미
CLK_HZ	50,000,000	메인 클록(다이얼 F)
PWM_HZ	1,000	깜빡임 주파수 1 kHz
LEVELS	10	밝기 11단계(0~10)
DEBOUNCE_CYCLES	1,000,000	20 ms 안정 판정

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1 (누르면 리셋)
button	N8	키패드 2 (밝기 한 단계 올림)
led[7:0]	L4 M4 M2 N7 M7 M3 M1 N5	LED1~LED8 (led[0] = LED8)

④ RTL

```
`timescale 1ns/1ps
module lab3_led_pwm #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer PWM_HZ = 1_000,
    parameter integer LEVELS = 10,
    parameter integer DEBOUNCE_CYCLES = 1_000_000
) (
    input wire clk_50mhz,
    input wire rst_p,
    input wire button,
    output wire [7:0] led
);

wire press;
wire pwm;
reg [3:0] level;

button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button (
    .clk(clk_50mhz), .rst_p(rst_p), .button(button), .pulse(press)
);

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p)
        level <= 4'd0;
    else if (press)
        level <= (level == LEVELS) ? 4'd0 : level + 1'b1;
end

pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm (
    .clk(clk_50mhz), .rst_p(rst_p), .level(level), .pwm(pwm)
```

```

);

    assign led = {8{pwm}};
endmodule

```

```

`timescale 1ns/1ps
module pwm_channel #(
    parameter integer PERIOD_CYCLES = 50_000,
    parameter integer LEVELS = 10
) (
    input  wire clk,
    input  wire rst_p,
    input  wire [3:0] level,
    output reg  pwm
);
    localparam integer COUNT_WIDTH = (PERIOD_CYCLES < 2) ? 1 : $clog2(PERIOD_CYCLES);
    reg [COUNT_WIDTH-1:0] count;
    integer threshold;

    always @* begin
        if (level >= LEVELS)
            threshold = PERIOD_CYCLES;
        else
            threshold = (PERIOD_CYCLES * level) / LEVELS;
    end

    always @(posedge clk or posedge rst_p) begin
        if (rst_p) begin
            count <= {COUNT_WIDTH{1'b0}};
            pwm <= 1'b0;
        end else begin
            pwm <= (count < threshold);
            if (count == PERIOD_CYCLES - 1)
                count <= {COUNT_WIDTH{1'b0}};
            else
                count <= count + 1'b1;
        end
    end
end
endmodule

```

```

`timescale 1ns/1ps
module button_onepulse #(
    parameter integer STABLE_CYCLES = 1_000_000
) (
    input  wire clk,
    input  wire rst_p,

```

```

input wire button,
output reg pulse
);

(* ASYNC_REG = "TRUE" *) reg button_meta;
(* ASYNC_REG = "TRUE" *) reg button_sync;
localparam integer COUNT_WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
reg [COUNT_WIDTH-1:0] stable_count;
reg accepted;

always @(posedge clk or posedge rst_p) begin
    if (rst_p) begin
        button_meta <= 1'b0;
        button_sync <= 1'b0;
    end else begin
        button_meta <= button;
        button_sync <= button_meta;
    end
end

always @(posedge clk or posedge rst_p) begin
    if (rst_p) begin
        stable_count <= {COUNT_WIDTH{1'b0}};
        accepted <= 1'b0;
        pulse <= 1'b0;
    end else begin
        pulse <= 1'b0;
        if (button_sync == accepted) begin
            stable_count <= {COUNT_WIDTH{1'b0}};
        end else if (stable_count == STABLE_CYCLES - 1) begin
            stable_count <= {COUNT_WIDTH{1'b0}};
            accepted <= button_sync;
            pulse <= button_sync;
        end else begin
            stable_count <= stable_count + 1'b1;
        end
    end
end
endmodule

```

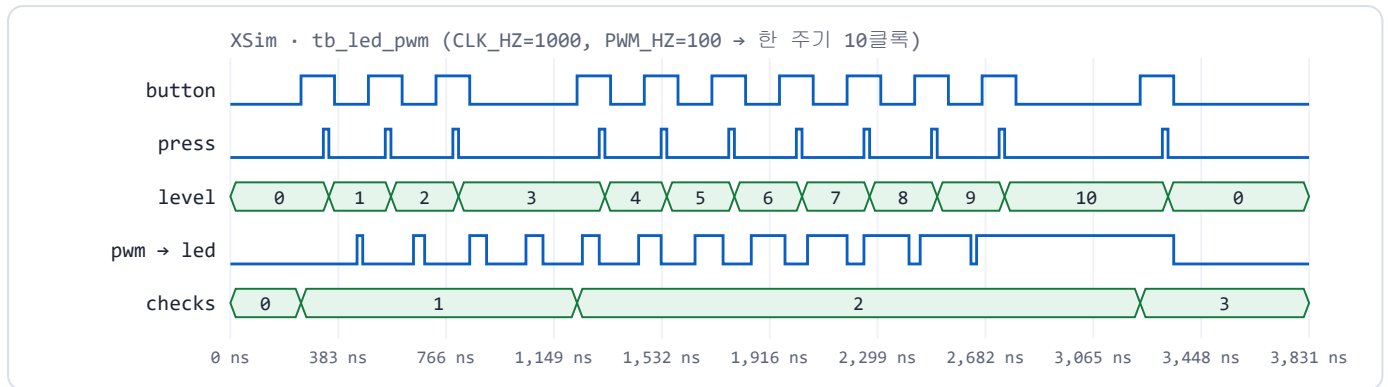
⑤ 시뮬레이션

50,000클럭짜리 주기를 그대로 돌리면 파형이 너무 길어진다. 테스트벤치는 파라미터만 줄여(CLK_HZ=1000, PWM_HZ=100) 한 주기를 10클럭으로 만들고, 한 주기 안의 High 개수가 단계 값과 같은지 센다. 영화를 빨리 감기로 보되, 줄거리는 그대로인 것과 같다.

PASS

Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_LED_PWM_PASS checks=4

검사 4개: 리셋 직후 0, 3번 눌러 3, 7번 더 눌러 10(항상 켜짐), 한 번 더 눌러 0으로 되돌아감.



버튼을 누를 때마다 `press` 가 한 클럭 뜨고 `level` 이 1씩 오른다. 10 다음에는 0으로 돌아가며, level 10에서는 `pwm`이 계속 High다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	92 / 48,000	0.19%
Slice Registers (FF)	52 / 96,000	0.05%
Bonded IOB	11 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클럭 1개
Setup WNS / Hold WHS	8.971 ns / 0.160 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	다이얼 F, 비트스트림 다운로드	LED 8개 모두 꺼짐 (L=0)	눈으로 확인
2	키패드 2를 한 번씩 누름	누를 때마다 8개가 같이 조금씩 밝아짐	영상 촬영
3	11번째 누름	완전히 꺼짐으로 돌아감	눈으로 확인
4	키패드 1	즉시 꺼짐	눈으로 확인

2. 실험 2 · RGB LED 색 조합 (3채널 PWM)

① 목적

빨강·초록·파랑 세 채널에 각자 PWM을 달아 버튼 3개로 섞는 비율을 바꾼다. 물감 세 통을 조금씩 섞어 원하는 색을 만드는 것과 같다. 실험 1의 `pwm_channel` 과 `button_onepulse` 를 그대로 세 번씩 재사용했다.

② 이론 · 설계 계산

$$D_R = \frac{L_R}{10}, \quad D_G = \frac{L_G}{10}, \quad D_B = \frac{L_B}{10} \quad (\text{각 } L \in \{0, \dots, 10\})$$

조합 가능한 색은 $11^3 = 1\,331$ 가지다. PWM 주파수는 실험 1과 같은 1 kHz($N = 50\,000$)라서 깜빡임은 보이지 않는다. 보드의 RGB LED 4개는 같은 색으로 함께 켜진다(`led_r = {4{pwm_r}}`).

③ 포트 · 보드 매핑

포트	핀	보드 장치
<code>clk_50mhz</code>	B6	MAIN_CLOCK1
<code>rst_p</code>	K4	키패드 1
<code>button_r / g / b</code>	N8 / N4 / N1	키패드 2 / 3 / 4
<code>led_r[3:0]</code>	T2 U1 P2 R3	RGB LED 4개의 R
<code>led_g[3:0]</code>	U5 V1 R7 T6	RGB LED 4개의 G
<code>led_b[3:0]</code>	U3 W2 R5 T3	RGB LED 4개의 B

핀 확인 대기 — 대상: RGB 12핀(T2 … T3). 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

④ RTL

```
`timescale 1ns/1ps
module lab3_rgb_pwm #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer PWM_HZ = 1_000,
    parameter integer LEVELS = 10,
    parameter integer DEBOUNCE_CYCLES = 1_000_000
) (
    input  wire clk_50mhz,
    input  wire rst_p,
    input  wire button_r,
    input  wire button_g,
    input  wire button_b,
    output wire [3:0] led_r,
    output wire [3:0] led_g,
    output wire [3:0] led_b
);
    wire press_r, press_g, press_b;
    wire pwm_r, pwm_g, pwm_b;

    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_r (
        .clk(clk_50mhz), .rst_p(rst_p), .button(button_r), .pulse(press_r));
    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_g (
        .clk(clk_50mhz), .rst_p(rst_p), .button(button_g), .pulse(press_g));
    button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_b (
        .clk(clk_50mhz), .rst_p(rst_p), .button(button_b), .pulse(press_b));

    reg [3:0] level_r, level_g, level_b;
    always @(posedge clk_50mhz or posedge rst_p) begin
        if (rst_p) begin
            level_r <= 0; level_g <= 0; level_b <= 0;
        end else begin
            if (press_r) level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;
            if (press_g) level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;
            if (press_b) level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
        end
    end

    pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_r (
        .clk(clk_50mhz), .rst_p(rst_p), .level(level_r), .pwm(pwm_r));
    pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_g (
        .clk(clk_50mhz), .rst_p(rst_p), .level(level_g), .pwm(pwm_g));
    pwm_channel #(.PERIOD_CYCLES(CLK_HZ / PWM_HZ), .LEVELS(LEVELS)) u_pwm_b (
        .clk(clk_50mhz), .rst_p(rst_p), .level(level_b), .pwm(pwm_b));

    assign led_r = {4{pwm_r}};
    assign led_g = {4{pwm_g}};
    assign led_b = {4{pwm_b}};
```

```

assign led_g = {4{pwm_g}};
assign led_b = {4{pwm_b}};
endmodule

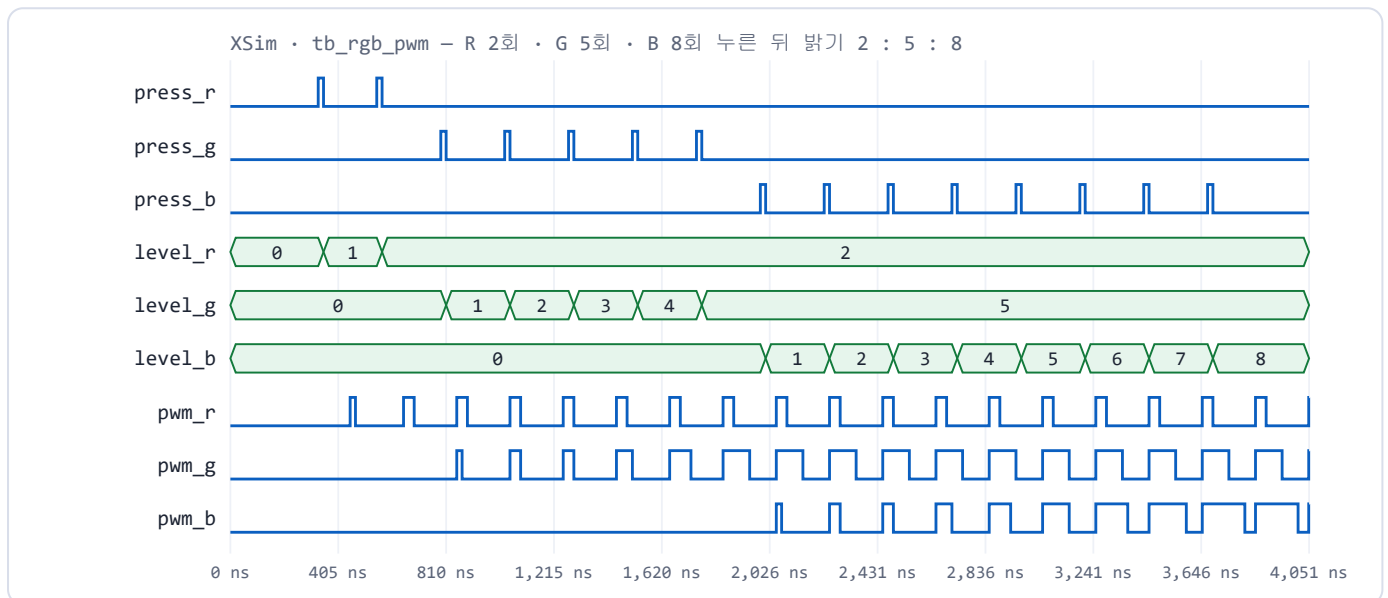
```

pwm_channel.v , button_onepulse.v 는 실험 1과 같은 파일이다.

⑤ 시뮬레이션

PASS Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_RGB_PWM_PASS checks=2

검사 2개: 리셋 직후 (0,0,0), R 2회·G 5회·B 8회 누른 뒤 한 주기(10클록) 안의 High 개수가 정확히 (2,5,8).



세 버튼이 서로 섞이지 않고 각자 자기 채널의 단계만 올린다. 아래 세 줄에서 High 폭이 2 : 5 : 8로 넓어지는 것이 보인다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	276 / 48,000	0.57%
Slice Registers (FF)	144 / 96,000	0.15%
Bonded IOB	17 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클록 1개
Setup WNS / Hold WHS	8.629 ns / 0.122 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	키패드 2만 여러 번	빨간색만 점점 밝아짐	영상
2	키패드 1 → 키패드 3만	초록색만	영상
3	R 10 · G 10 · B 0	노란색 계열	눈으로 확인
4	R 10 · G 10 · B 10	흰색 계열	눈으로 확인

3. 실험 3 · 피에조 부저 (주파수 분주로 음 만들기)

① 목적

50 MHz 클럭을 나눠 294 Hz 사각파를 만들고 피에조 부저로 "레(D4)" 음을 낸다. 줄넘기를 일정한 박자로 넘으면 줄이 같은 소리를 반복하듯, 1초에 294번 떨어지는 신호가 그 높이의 소리가 된다.

② 이론 · 설계 계산

출력을 반주기마다 뒤집으므로 반주기 클럭 수는 다음과 같다(정수 나눗셈).

$$H = \left\lfloor \frac{f_{clk}}{2f_{tone}} \right\rfloor = \left\lfloor \frac{50\,000\,000}{588} \right\rfloor = 85\,034$$

$$f_{out} = \frac{f_{clk}}{2H} = \frac{50\,000\,000}{170\,068} \approx 294.0000 \text{ Hz}$$

평균율의 D4는 $440 \times 2^{-7/12} \approx 293.66$ Hz이므로 차이는 약 0.12%(2 cent)로 귀로는 구별되지 않는다.

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1 (누르고 있는 동안 소리 멈춤)
piezo	Y21	피에조 부저

핀 확인 대기 — 대상: 피에조 핀 Y21. 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

④ RTL

```
`timescale 1ns/1ps
module lab3_piezo #(
    parameter integer CLK_HZ = 50_000_000,
```

```

parameter integer TONE_HZ = 294
) (
    input wire clk_50mhz,
    input wire rst_p,
    output reg piezo
);
localparam integer HALF_PERIOD = CLK_HZ / (2 * TONE_HZ);
localparam integer COUNT_WIDTH = (HALF_PERIOD < 2) ? 1 : $clog2(HALF_PERIOD);
reg [COUNT_WIDTH-1:0] count;

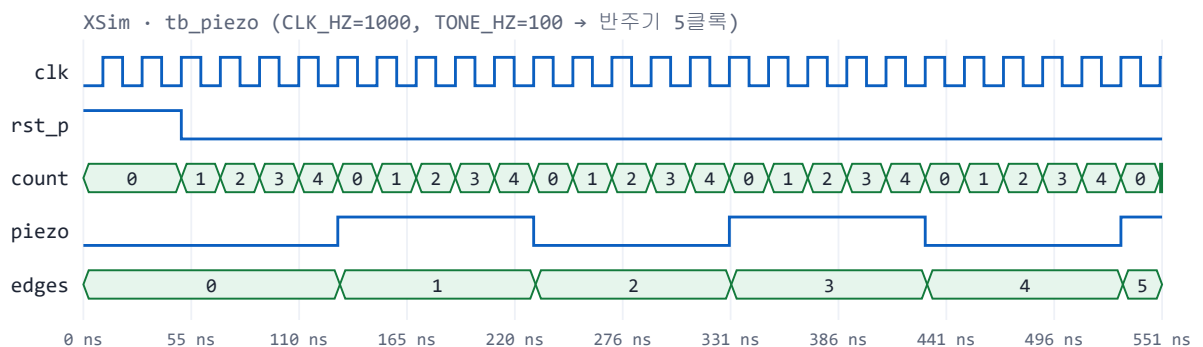
always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        count <= 0;
        piezo <= 0;
    end else if (count == HALF_PERIOD - 1) begin
        count <= 0;
        piezo <= ~piezo;
    end else begin
        count <= count + 1'b1;
    end
end
endmodule

```

⑤ 시뮬레이션

PASS Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_PIEZO_PASS edges=5

TONE_HZ=100, CLK_HZ=1000으로 줄이면 $H = 5$. 25클록 동안 count 가 0이 되는 간격이 매번 정확히 5클록 인지 확인했다.



count가 0→4를 돌 때마다 piezo가 뒤집힌다. 한 주기 = 10클록 = 200 ns.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	21 / 48,000	0.04%
Slice Registers (FF)	18 / 96,000	0.02%
Bonded IOB	3 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클록 1개
Setup WNS / Hold WHS	16.761 ns / 0.246 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	PIEZO ENABLE 스위치 ON, 다운로드	일정한 "레" 음이 계속 남	소리 녹음(영상)
2	키패드 1을 누르고 있음	소리가 멈춤	귀로 확인
3	(선택) Y21 신호를 오실로스코프로 측정	주기 3.401 ms, 294 Hz 사각파	오실로 사진

4. 실험 4 · 스텝 모터 (2상 여자 순서 만들기)

① 목적

스텝 모터 코일 4개에 정해진 순서로 전류를 넣어 한 칸씩 돌린다. 계단을 한 칸씩 오르듯 순서를 앞으로 돌리면 정 방향, 뒤로 돌리면 역방향이 된다.

② 이론 · 설계 계산

항상 인접한 두 상을 함께 켜는 2상 여자(full step) 순서를 쓴다.

state	stepmotor[3:0]	켜진 상
0	0011	A · B
1	0110	B · C
2	1100	C · D
3	1001	D · A

$$T_{step} = \frac{f_{clk}}{f_{step}} = \frac{50\,000\,000}{100} = 500\,000 \text{ 클록} = 10 \text{ ms}$$

1초에 100스텝이다. 스텝각이 1.8°인 모터라면 $1.8^{\circ} \times 100 = 180^{\circ}/s = 30 \text{ rpm}$ 이다(보드 모터의 스텝각은 매뉴얼로 확인할 것).

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1
enable	N8	키패드 2 (누르고 있는 동안 회전)
direction	N4	키패드 3 (누르고 있으면 역방향)
stepmotor[3:0]	Y20 Y22 AA20 AA21	스텝 모터 드라이버 입력

핀 확인 대기 — 대상: 스텝 모터 4핀(Y20 Y22 AA20 AA21)과 상 순서. 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

조작 편의 — 키패드는 누르는 동안만 High인 푸시 버튼이다. 따라서 모터는 키패드 2를 누르고 있는 동안만 돈다. 계속 돌리는 모습을 촬영하려면 enable·direction을 DIP 스위치로 옮기는 방법도 있다(XDC만 수정).

④ RTL

```
`timescale 1ns/1ps
module lab3_stepper #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer STEP_HZ = 100
) (
    input wire clk_50mhz,
    input wire rst_p,
    input wire enable,
    input wire direction,
    output reg [3:0] stepmotor
);
    localparam integer STEP_CYCLES = CLK_HZ / STEP_HZ;
    localparam integer COUNT_WIDTH = (STEP_CYCLES < 2) ? 1 : $clog2(STEP_CYCLES);
    reg [COUNT_WIDTH-1:0] count;
    reg [1:0] state;
    (* ASYNC_REG = "TRUE" *) reg enable_meta, enable_sync;
    (* ASYNC_REG = "TRUE" *) reg direction_meta, direction_sync;

    always @(posedge clk_50mhz or posedge rst_p) begin
        if (rst_p) begin
```

```

        enable_meta <= 0; enable_sync <= 0;
        direction_meta <= 0; direction_sync <= 0;
    end else begin
        enable_meta <= enable; enable_sync <= enable_meta;
        direction_meta <= direction; direction_sync <= direction_meta;
    end
end

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        count <= 0;
        state <= 0;
    end else if (!enable_sync) begin
        count <= 0;
    end else if (count == STEP_CYCLES - 1) begin
        count <= 0;
        state <= direction_sync ? state - 1'b1 : state + 1'b1;
    end else begin
        count <= count + 1'b1;
    end
end

always @* begin
    case (state)
        2'd0: stepmotor = 4'b0011;
        2'd1: stepmotor = 4'b0110;
        2'd2: stepmotor = 4'b1100;
        default: stepmotor = 4'b1001;
    endcase
end
endmodule

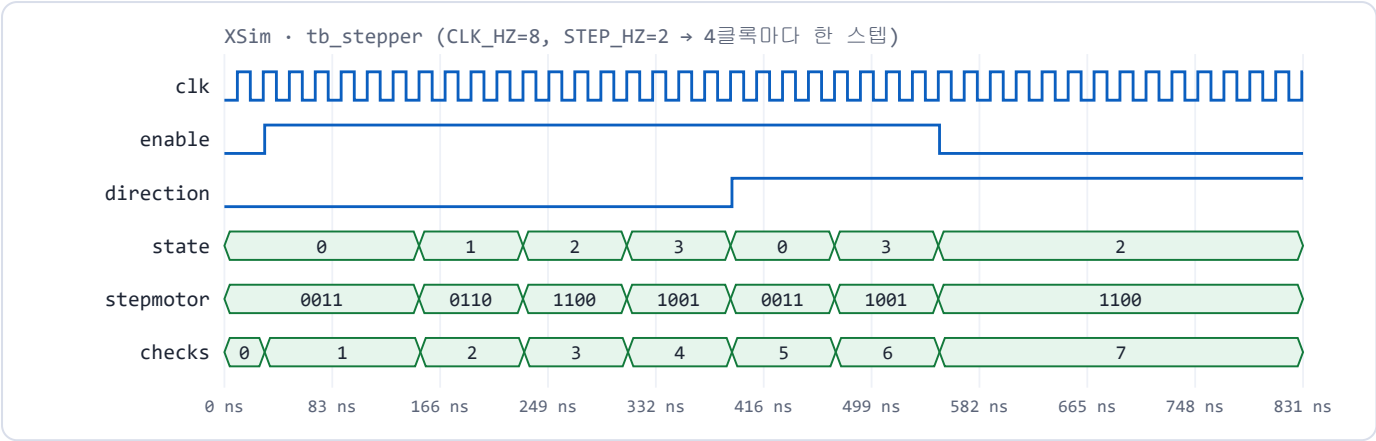
```

⑤ 시뮬레이션

PASS

Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_STEPPER_PASS checks=8

검사 8개: 리셋값 0011 → 정방향 0110·1100·1001·0011 → 역방향 1001·1100 → enable을 내린 뒤 12클록 동안 1100 유지.



direction이 1이 되면 state가 0→3→2로 거꾸로 간다. enable이 0이면 state가 멈춘다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	20 / 48,000	0.04%
Slice Registers (FF)	25 / 96,000	0.03%
Bonded IOB	8 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클록 1개
Setup WNS / Hold WHS	16.252 ns / 0.160 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	키패드 2를 누르고 있음	일정 속도로 한 방향 회전	영상
2	키패드 2+3을 함께 누름	반대 방향 회전	영상
3	손을 땀	그 자리에서 정지(코일 여자 유지)	눈으로 확인

5. 실험 5 · MM:SS 디지털 시계 (7세그먼트 4자리)

① 목적

1초마다 올라가는 분·초 시계를 만들고 7세그먼트 4자리에 **MM.SS**로 표시한다(분 1의 자리의 점을 구분자로 사용). 네 자리를 번갈아 아주 빨리 켜서, 손전등 네 개를 돌려 비추는데도 모두 켜져 보이는 것처럼 표시한다.

② 이론 · 설계 계산

$$1\text{ s} = f_{clk} \text{ 클록} = 50\,000\,000 \text{ 클록}$$

$$T_{digit} = \frac{f_{clk}}{f_{scan}} = \frac{50\,000\,000}{4\,000} = 12\,500 \text{ 클록} = 250\ \mu\text{s}, \quad T_{frame} = 4T_{digit} = 1\text{ ms}$$

한 자리가 1 ms마다 다시 켜지므로(자리당 1 kHz) 깜빡임은 보이지 않는다. 카운터는 초 1의 자리 9 → 0일 때 초 10의 자리로, 59초 → 00초일 때 분으로 올림하며 59:59 다음은 00:00이다(주기 3,600 s).

세그먼트 비트 순서는 LAB2-08(보드 검증 완료)과 같은 `seg_data[7:0] = {a,b,c,d,e,f,g,dp}` , High = 켜짐이다. 자리 선택 `seg_com` 은 Low = 선택이며 COM[3]~COM[0]을 쓴다.

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1 (00.00으로)
seg_data[0..7]	H2 J7 J3 J1 E4 E2 F5 F1	세그먼트 dp g f e d c b a (Active-High)
seg_com[0..3]	K5 K3 K1 L6	COM0~COM3 (Active-Low)
seg_com[4..7]	G3 G1 H6 H4	사용 안 함 (항상 High = 꺼짐)

④ RTL

```
`timescale 1ns/1ps
module lab3_mmss_clock #(
```

```

parameter integer CLK_HZ = 50_000_000,
parameter integer SCAN_HZ = 4_000
) (
    input wire clk_50mhz,
    input wire rst_p,
    output reg [7:0] seg_data,
    output reg [7:0] seg_com
);
wire [3:0] mt, mo, st, so;
wire [7:0] seg_mt, seg_mo, seg_st, seg_so;
localparam integer SCAN_CYCLES = CLK_HZ / SCAN_HZ;
localparam integer SCAN_WIDTH = (SCAN_CYCLES < 2) ? 1 : $clog2(SCAN_CYCLES);
reg [SCAN_WIDTH-1:0] scan_count;
reg [1:0] scan_select;

mmss_counter #(.CLK_HZ(CLK_HZ)) u_counter (
    .clk(clk_50mhz), .rst_p(rst_p), .minute_tens(mt), .minute_ones(mo),
    .second_tens(st), .second_ones(so));
sevenseg_decode u_mt(.digit(mt), .segments(seg_mt));
sevenseg_decode u_mo(.digit(mo), .segments(seg_mo));
sevenseg_decode u_st(.digit(st), .segments(seg_st));
sevenseg_decode u_so(.digit(so), .segments(seg_so));

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        scan_count <= 0;
        scan_select <= 0;
    end else if (scan_count == SCAN_CYCLES - 1) begin
        scan_count <= 0;
        scan_select <= scan_select + 1'b1;
    end else begin
        scan_count <= scan_count + 1'b1;
    end
end

always @* begin
    seg_com = 8'b1111_1111;
    seg_data = 8'b0000_0000;
    case (scan_select)
        2'd0: begin seg_com = 8'b1111_0111; seg_data = seg_mt; end
        2'd1: begin seg_com = 8'b1111_1011; seg_data = seg_mo | 8'b0000_0001; end
        2'd2: begin seg_com = 8'b1111_1101; seg_data = seg_st; end
        default: begin seg_com = 8'b1111_1110; seg_data = seg_so; end
    endcase
end
endmodule

```

```

`timescale 1ns/1ps
module mmss_counter #(
    parameter integer CLK_HZ = 50_000_000
) (
    input wire clk,
    input wire rst_p,
    output reg [3:0] minute_tens,
    output reg [3:0] minute_ones,
    output reg [3:0] second_tens,
    output reg [3:0] second_ones
);
    localparam integer COUNT_WIDTH = (CLK_HZ < 2) ? 1 : $clog2(CLK_HZ);
    reg [COUNT_WIDTH-1:0] subsecond;

    always @(posedge clk or posedge rst_p) begin
        if (rst_p) begin
            subsecond <= 0;
            minute_tens <= 0; minute_ones <= 0;
            second_tens <= 0; second_ones <= 0;
        end else if (subsecond == CLK_HZ - 1) begin
            subsecond <= 0;
            if (second_ones != 9)
                second_ones <= second_ones + 1'b1;
            else begin
                second_ones <= 0;
                if (second_tens != 5)
                    second_tens <= second_tens + 1'b1;
                else begin
                    second_tens <= 0;
                    if (minute_ones != 9)
                        minute_ones <= minute_ones + 1'b1;
                    else begin
                        minute_ones <= 0;
                        if (minute_tens != 5)
                            minute_tens <= minute_tens + 1'b1;
                        else
                            minute_tens <= 0;
                    end
                end
            end
        end else begin
            subsecond <= subsecond + 1'b1;
        end
    end
endmodule

```

```

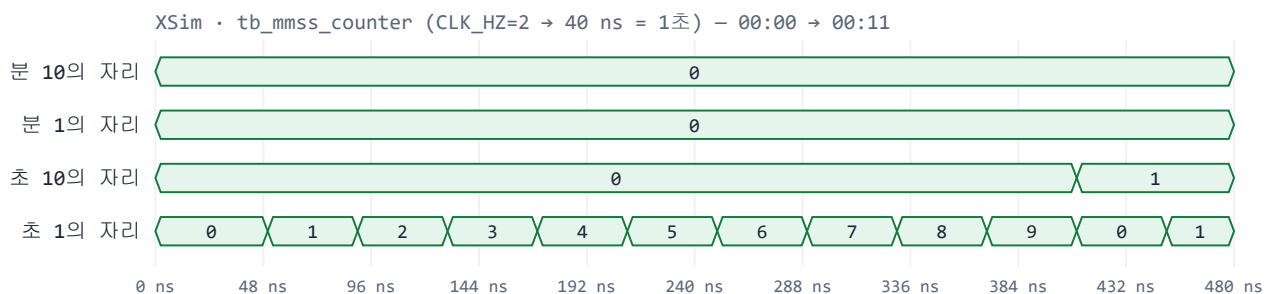
`timescale 1ns/1ps
module sevenseg_decode(
    input wire [3:0] digit,
    output reg [7:0] segments
);
    always @* begin
        case (digit)
            4'd0: segments = 8'b1111_1100;
            4'd1: segments = 8'b0110_0000;
            4'd2: segments = 8'b1101_1010;
            4'd3: segments = 8'b1111_0010;
            4'd4: segments = 8'b0110_0110;
            4'd5: segments = 8'b1011_0110;
            4'd6: segments = 8'b1011_1110;
            4'd7: segments = 8'b1110_0000;
            4'd8: segments = 8'b1111_1110;
            4'd9: segments = 8'b1111_0110;
            default: segments = 8'b0000_0000;
        endcase
    end
endmodule

```

⑤ 시뮬레이션

PASS Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_MMSS_PASS checks=7

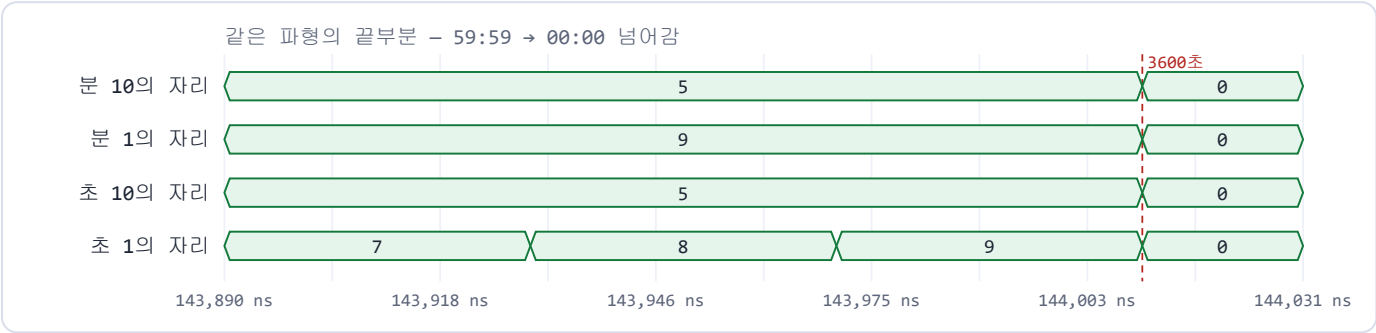
CLK_HZ=2로 줄여 2클록(40 ns)을 1초로 만들었다. 검사 7개: 디코더 0~9, 리셋 00:00, 10초 뒤 00:10, 60초 뒤 01:00, 3,599초 뒤 59:59, 1초 더 뒤 00:00.



초 1의 자리가 9 다음 0이 되면서 초 10의 자리가 1 오른다.



00:59 → 01:00에서 초 두 자리가 0이 되고 분 1의 자리가 오른다.



3,600초째에 네 자리가 동시에 0으로 넘어간다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	83 / 48,000	0.17%
Slice Registers (FF)	58 / 96,000	0.06%
Bonded IOB	18 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클록 1개
Setup WNS / Hold WHS	15.118 ns / 0.210 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	다운로드	오른쪽 4자리에 00.00부터 1초마다 증가	영상
2	스톱워치와 1분 비교	오차 1초 미만	영상에 스톱워치 함께 촬영
3	키패드 1	00.00으로 되돌아감	눈으로 확인
4	자리 순서 확인	왼쪽부터 분10·분1·초10·초1	순서가 뒤집히면 seg_com 매핑 수정

6. 실험 6 · Character LCD (HD44780 호환 16×2)

① 목적

16×2 문자 LCD를 초기화하고 첫 줄에 FPGA LAB3 , 둘째 줄에 LCD CONTROLLER 를 쓴다. LCD는 명령을 받은 뒤 처리할 시간이 필요하므로, 정해진 순서와 기다리는 시간을 지키는 "순서표 실행기"(상태 머신)로 만들었다. 요 리 레시피처럼 순서와 기다리는 시간이 정해져 있다.

② 이론 · 설계 계산

10 μs짜리 tick(500클록)을 기본 단위로 쓴다.

구간	tick	실제 시간	HD44780 요구
전원 안정 대기	2,000	20 ms	≥ 15 ms
E High 폭	1	10 μs	≥ 0.23 μs
일반 명령·문자 후 대기	4	40 μs	≥ 37 μs
Clear(0x01) 후 대기	200	2 ms	≥ 1.52 ms

한 바이트는 준비(1) → E High(1) → E Low(1) → 대기(4) → 다음으로(1) = 8 tick = 80 μs다. 초기화가 끝난 뒤 index 6~39(34바이트)를 계속 다시 쓰므로 화면 전체가 약 $34 \times 80 \mu s = 2.72 ms$ 마다 새로 쓰인다.

index	RS	DATA	의미
0~2	0	0x38	8비트 버스 · 2줄 · 5×8 글꼴 (3회)
3	0	0x0C	화면 ON, 커서 OFF
4	0	0x06	쓰고 나면 커서 오른쪽 이동
5	0	0x01	화면 지우기(2 ms 대기)
6	0	0x80	첫 줄 첫 칸으로
7~22	1	"FPGA LAB3" + 공백	첫 줄 16칸
23	0	0xC0	둘째 줄 첫 칸으로
24~39	1	"LCD CONTROLLER" + 공백	둘째 줄 16칸

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1
lcd_e / lcd_rs / lcd_rw	A6 / G6 / D6	LCD E · RS · RW (RW는 항상 0 = 쓰기)
lcd_data[0..7]	A4 B2 C3 D4 A2 C5 C1 D1	LCD D0~D7

핀 확인 대기 — 대상: LCD 11핀(A6 G6 D6, A4 … D1). 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

④ RTL

```
`timescale 1ns/1ps
module lab3_character_lcd #(
    parameter integer TICK_CYCLES = 500,
    parameter integer POWER_TICKS = 2_000,
    parameter integer NORMAL_WAIT_TICKS = 4,
    parameter integer CLEAR_WAIT_TICKS = 200
```

```

) (
    input wire clk_50mhz,
    input wire rst_p,
    output reg lcd_e,
    output reg lcd_rs,
    output wire lcd_rw,
    output reg [7:0] lcd_data
);

localparam integer TICK_WIDTH = (TICK_CYCLES < 2) ? 1 : $clog2(TICK_CYCLES);
reg [TICK_WIDTH-1:0] tick_count;
reg tick;
reg [1:0] phase;
reg [5:0] index;
integer wait_count;
integer power_count;
reg byte_rs;
reg [7:0] byte_data;
assign lcd_rw = 1'b0;

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        tick_count <= 0;
        tick <= 0;
    end else if (tick_count == TICK_CYCLES - 1) begin
        tick_count <= 0;
        tick <= 1;
    end else begin
        tick_count <= tick_count + 1'b1;
        tick <= 0;
    end
end

always @* begin
    byte_rs = 1'b0;
    byte_data = 8'h00;
    case (index)
        0, 1, 2: byte_data = 8'h38;
        3: byte_data = 8'h0c;
        4: byte_data = 8'h06;
        5: byte_data = 8'h01;
        6: byte_data = 8'h80;
        7: begin byte_rs=1; byte_data="F"; end
        8: begin byte_rs=1; byte_data="P"; end
        9: begin byte_rs=1; byte_data="G"; end
        10: begin byte_rs=1; byte_data="A"; end
        11: begin byte_rs=1; byte_data=" "; end
        12: begin byte_rs=1; byte_data="L"; end
    endcase
end

```

```

13: begin byte_rs=1; byte_data="A"; end
14: begin byte_rs=1; byte_data="B"; end
15: begin byte_rs=1; byte_data="3"; end
16,17,18,19,20,21,22: begin byte_rs=1; byte_data=" "; end
23: byte_data = 8'hc0;
24: begin byte_rs=1; byte_data="L"; end
25: begin byte_rs=1; byte_data="C"; end
26: begin byte_rs=1; byte_data="D"; end
27: begin byte_rs=1; byte_data=" "; end
28: begin byte_rs=1; byte_data="C"; end
29: begin byte_rs=1; byte_data="O"; end
30: begin byte_rs=1; byte_data="N"; end
31: begin byte_rs=1; byte_data="T"; end
32: begin byte_rs=1; byte_data="R"; end
33: begin byte_rs=1; byte_data="O"; end
34: begin byte_rs=1; byte_data="L"; end
35: begin byte_rs=1; byte_data="L"; end
36: begin byte_rs=1; byte_data="E"; end
37: begin byte_rs=1; byte_data="R"; end
38,39: begin byte_rs=1; byte_data=" "; end
default: byte_data = 8'h00;
endcase
end

always @(posedge clk_50mhz or posedge rst_p) begin
    if (rst_p) begin
        lcd_e <= 0; lcd_rs <= 0; lcd_data <= 0;
        phase <= 0; index <= 0; wait_count <= 0; power_count <= 0;
    end else if (tick) begin
        if (power_count < POWER_TICKS) begin
            power_count <= power_count + 1;
            lcd_e <= 0;
        end else begin
            case (phase)
                0: begin
                    lcd_e <= 0; lcd_rs <= byte_rs; lcd_data <= byte_data;
                    phase <= 1;
                end
                1: begin lcd_e <= 1; phase <= 2; end
                2: begin
                    lcd_e <= 0;
                    wait_count <= (index == 5) ? CLEAR_WAIT_TICKS : NORMAL_WAIT_TICKS;
                    phase <= 3;
                end
                default: begin
                    if (wait_count > 0)
                        wait_count <= wait_count - 1;
                end
            endcase
        end
    end
end

```

```

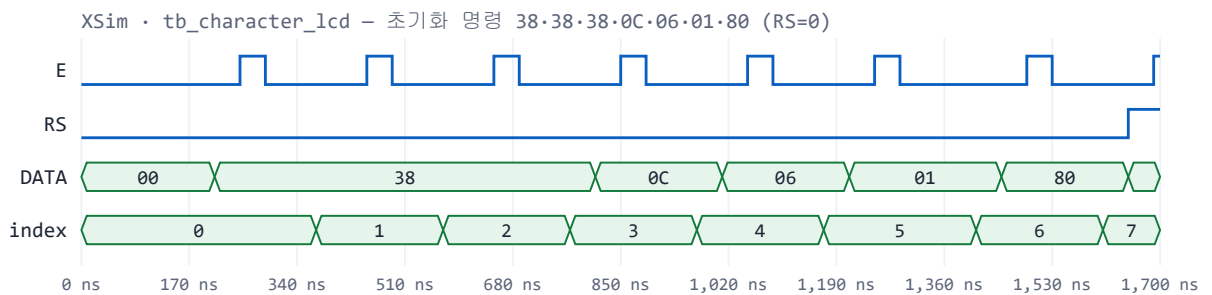
else begin
    index <= (index == 39) ? 6 : index + 1'b1;
    phase <= 0;
end
end
endcase
end
end
end
endmodule

```

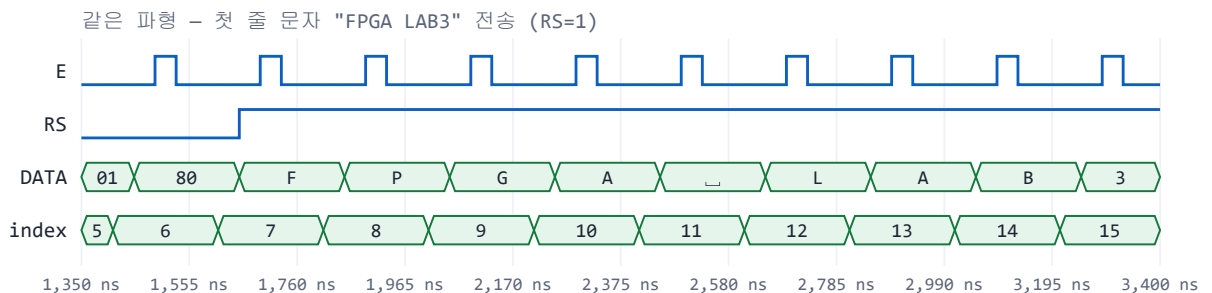
⑤ 시뮬레이션

PASS Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_LCD_PASS bytes=40

tick을 2클록으로 줄이고, E의 하강 에지마다 RS·DATA·RW를 기대값 표와 비교했다. 40바이트(초기화 7 + 문자·주소 33) 모두 일치.



초기화 구간. RS=0에서 38·38·38·0C·06·01·80이 차례로 나가며, 01(Clear) 뒤에는 대기가 길다.



RS=1로 바뀐 뒤 F·P·G·A·space·L·A·B·3이 E 펄스마다 한 글자씩 나간다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	111 / 48,000	0.23%
Slice Registers (FF)	93 / 96,000	0.10%
Bonded IOB	13 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클록 1개
Setup WNS / Hold WHS	14.220 ns / 0.167 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	다운로드	첫 줄 FPGA LAB3, 둘째 줄 LCD CONTROLLER	사진
2	글자가 안 보이면 명암(가변저항) 조절	글자가 선명해짐	사진
3	키패드 1을 눌렀다 땀	잠깐 멈췄다가 다시 같은 글자 표시	영상

7. 실험 7 · UART 에코 (9600 bps, 8N1)

① 목적

PC 터미널에서 보낸 글자를 받아 그대로 돌려보내고(에코), 받은 글자의 ASCII 값을 LED 8개에 2진수로 보여준다. UART는 약속한 빠르기(보레이트)로 한 비트씩 차례로 보내는 방식이라, 보내는 쪽과 받는 쪽이 같은 박자로 손뼉을 치는 것과 같다.

② 이론 · 설계 계산

한 비트에 해당하는 클록 수를 반올림으로 구한다.

$$\text{DIV} = \left\lceil \frac{f_{clk} + \text{BAUD}/2}{\text{BAUD}} \right\rceil = \left\lceil \frac{50\,004\,800}{9\,600} \right\rceil = 5\,208$$

$$\text{실제 보레이트} = \frac{50\,000\,000}{5\,208} \approx 9\,600.61 \text{ bps}, \quad \text{오차} \approx +0.0064\%$$

UART는 한 글자(10비트) 동안 오차가 쌓여도 반 비트(5%) 안이면 되므로 충분히 작다. 한 글자 프레임은 시작(0) + 데이터 8비트(LSB 먼저) + 정지(1) = 10비트 = 약 1.04 ms다. 수신기는 시작 비트의 하강 에지를 본 뒤 DIV/2만큼 기다려 비트 한가운데에서 표본을 잡는다.

③ 포트 · 보드 매핑

포트	핀	보드 장치
clk_50mhz	B6	MAIN_CLOCK1
rst_p	K4	키패드 1
uart_rxd	C6	PC → FPGA (USB-UART)
uart_txd	F6	FPGA → PC
led[7:0]	L4 M4 M2 N7 M7 M3 M1 N5	마지막 수신 바이트 (LED1 = MSB)

핀 확인 대기 — 대상: UART 2핀(C6, F6)과 RX/TX 방향. 매뉴얼의 장치별 핀 구성표와 아직 대조하지 못했다. 보드 실습 전에 확인하고, 다르면 XDC만 고친 뒤 다시 빌드한다.

④ RTL

```
`timescale 1ns/1ps
module lab3_uart_echo #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer BAUD = 9_600
) (
    input wire clk_50mhz,
    input wire rst_p,
    input wire uart_rxd,
    output wire uart_txd,
    output wire [7:0] led
);
    localparam integer DIV = (CLK_HZ + BAUD/2) / BAUD;
    wire [7:0] rx_data;
    wire rx_valid;
    wire rx_framing_error;
    wire tx_ready;
    reg tx_valid;
    reg [7:0] tx_data;
    reg [7:0] last_data;

    uart_rx #(.DIV(DIV)) u_rx(
        .clk(clk_50mhz), .rst_p(rst_p), .rx(uart_rxd), .data(rx_data),
        .valid(rx_valid), .framing_error(rx_framing_error));
    uart_tx #(.DIV(DIV)) u_tx(
        .clk(clk_50mhz), .rst_p(rst_p), .valid(tx_valid), .data(tx_data),
        .ready(tx_ready), .tx(uart_txd));

    always @(posedge clk_50mhz or posedge rst_p) begin
        if (rst_p) begin
            tx_valid <= 0; tx_data <= 0; last_data <= 0;
        end else begin
            tx_valid <= 0;
            if (rx_valid) begin
                last_data <= rx_data;
                if (tx_ready) begin
                    tx_data <= rx_data;
                    tx_valid <= 1;
                end
            end
        end
        assign led = last_data;
    end
endmodule
```

```

`timescale 1ns/1ps
module uart_rx #(parameter integer DIV = 5208)(
    input wire clk, rst_p, rx,
    output reg [7:0] data,
    output reg valid,
    output reg framing_error
);
    (* ASYNC_REG="TRUE" *) reg [1:0] sync;
    reg [1:0] state;
    integer timer;
    reg [2:0] bitno;
    reg [7:0] shift;
    always @(posedge clk) begin
        if (rst_p) begin
            sync<=3; state<=0; timer<=0; bitno<=0; shift<=0;
            data<=0; valid<=0; framing_error<=0;
        end else begin
            sync<={sync[0],rx}; valid<=0; framing_error<=0;
            case(state)
                0: if(!sync[1]) begin timer<=DIV/2-1; state<=1; end
                1: if(timer!=0) timer<=timer-1;
                   else if(sync[1]) state<=0;
                   else begin timer<=DIV-1; bitno<=0; state<=2; end
                2: if(timer!=0) timer<=timer-1;
                   else begin
                       shift[bitno]<=sync[1]; timer<=DIV-1;
                       if(bitno==7) state<=3; else bitno<=bitno+1'b1;
                   end
                3: if(timer!=0) timer<=timer-1;
                   else begin
                       if(sync[1]) begin data<=shift; valid<=1; end
                       else framing_error<=1;
                       state<=0;
                   end
            endcase
        end
    end
endmodule

module uart_tx #(parameter integer DIV = 5208)(
    input wire clk, rst_p, valid,
    input wire [7:0] data,
    output wire ready,
    output wire tx
);
    reg [9:0] shift;
    reg [3:0] remaining;

```

```

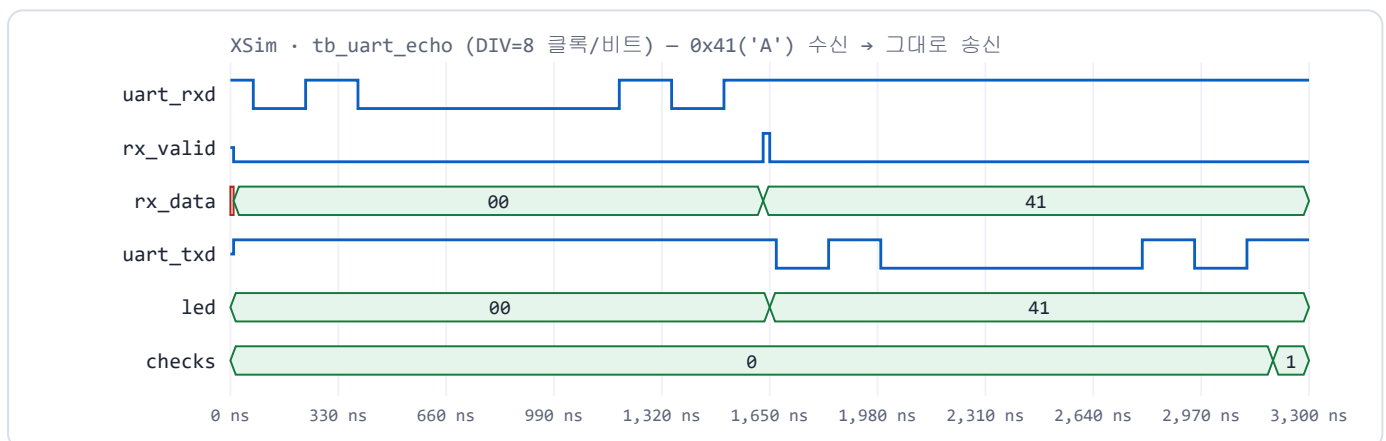
integer timer;
assign ready = (remaining == 0);
assign tx = ready ? 1'b1 : shift[0];
always @(posedge clk) begin
    if(rst_p) begin shift<=10'h3ff; remaining<=0; timer<=0; end
    else if(ready) begin
        if(valid) begin shift<={1'b1,data,1'b0}; remaining<=10; timer<=DIV-1; end
        end else if(timer!=0) timer<=timer-1;
        else begin timer<=DIV-1; shift<={1'b1,shift[9:1]}; remaining<=remaining-1'b1; end
    end
endmodule

```

⑤ 시뮬레이션

PASS Vivado XSim과 Icarus Verilog 12.0에서 같은 결과를 확인했다: LAB3_UART_ECHO_PASS checks=3

DIV=8로 줄여 0x41(A), 0x5A(Z), 0x0A(줄바꿈)를 보내고, 되돌아온 직렬 비트와 LED 값이 같은지 확인했다. 프레임 오류 0회.



rxdata에 A(0x41)가 들어온 뒤 rx_valid가 한 클럭 뜨고, 곧바로 txd로 같은 비트열이 나간다. LED는 41로 바뀐다.

⑥ 합성 · 구현 결과 (사전 빌드)

항목	값	비고
Slice LUTs	134 / 48,000	0.28%
Slice Registers (FF)	118 / 96,000	0.12%
Bonded IOB	12 / 338	XDC의 포트 수와 일치
BUFGCTRL	1 / 32	50 MHz 클럭 1개
Setup WNS / Hold WHS	14.617 ns / 0.131 ns	둘 다 양수 → 타이밍 충족

⑦ 보드 실습 계획

#	조작	기대 결과	확인 방법
1	PC 터미널(9600, 8N1, 흐름제어 없음) 연결	—	장치 관리자에서 COM 번호 확인
2	A 입력	화면에 A가 다시 찍힘, LED = 0100_0001	화면 캡처 + LED 사진
3	여러 글자 빠르게 입력	모두 그대로 에코	화면 캡처

참고

- 실습 매뉴얼 「Verilog HDL을 이용한 디지털 논리 회로 설계」 CH2 — 클록 선택표, 키패드 · LED · BDIO · 확장 헤더 핀 구성표
- HD44780U 데이터시트 — 초기화 순서와 명령 실행 시간
- LAB2-08 7세그먼트 자리 스캔 (보드 검증 완료) — 세그먼트 비트 순서와 COM 극성